

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' test (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: XP-14 — Página Web Metodologías TI

1. CONTEXTO

Se requiere una página web informativa/editorial sobre metodologías de informática y TI. El stack está explícitamente definido por el cliente: HTML5, CSS y JS puro. La página tiene 3 secciones de contenido (Home, Historias, Noticias) más secciones adicionales derivadas de los criterios de aceptación (FAQ, aviso de cookies). No hay backend, no hay autenticación, no hay base de datos. El contenido es estático o semi-estático. El entorno de ejecución final es un navegador vía protocolo HTTP/HTTPS (servidor web estático o CDN).

2. DECISIÓN ARQUITECTÓNICA

****Arquitectura elegida: Single-File Application (SFA) con separación mínima de assets.****

Un único `index.html` que actúa como SPA simulada mediante navegación por anclas y manipulación del DOM con JS vanilla. CSS en un archivo externo (`styles.css`) y JS en un único archivo plano (`main.js`). NO se usará `type='module'` ni imports ES6 nativos para garantizar compatibilidad máxima sin servidor de desarrollo.

****Tecnologías seleccionadas:****

- HTML5 semántico (nav, main, section, article, footer) !' WCAG 2.1 AA
- CSS3 con variables custom + media queries !' Responsividad
- JavaScript ES5/ES6 vanilla (sin frameworks) !' Sin dependencias de build
- DOMPurify (CDN, versión fija) !' Sanitización XSS en inputs de búsqueda/comentarios
- Google Analytics 4 (snippet inline) !' Criterio de Aceptación 5
- Meta CSP via `<meta http-equiv='Content-Security-Policy'>` !' Seguridad XSS

3. JUSTIFICACIÓN YAGNI

- ****¿Por qué NO React/Vue/Angular?*** El requerimiento es contenido editorial estático. Un framework SPA añadiría ~100KB+ de bundle, tiempo de hidratación y complejidad de build sin ningún beneficio funcional para este caso de uso.
- ****¿Por qué NO múltiples páginas HTML?*** Una SPA simulada con secciones visibles/ocultas via JS permite transiciones suaves, mantiene el estado del aviso de cookies y reduce peticiones HTTP, cumpliendo el criterio de carga <3 segundos.
- ****¿Por qué NO un SSG (Hugo, Eleventy)?*** El cliente especificó HTML5/CSS/JS directamente. Introducir una cadena de build viola YAGNI y añade fricción operacional sin valor demostrable para un MVP de 3 secciones.
- ****¿Por qué NO backend/Node.js?*** No hay persistencia de datos, autenticación ni lógica de servidor requerida. El contenido de Historias y Noticias se renderiza desde arrays JS en el cliente.
- ****¿Por qué DOMPurify desde CDN y no implementación propia?*** La sanitización XSS manual es propensa a errores. DOMPurify es la librería estándar de la industria, auditada y mantenida. Se fija a una versión específica para evitar supply-chain attacks.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- ****Riesgo 1 (BAJO):*** El contenido de Noticias e Historias está hardcodeado en JS. Si el cliente necesita actualizar contenido sin tocar código en el futuro, se requerirá migrar a un CMS headless o JSON externo. Mitigación: estructurar los datos como arrays de objetos en un bloque claramente delimitado en `main.js` para facilitar extracción futura.
- ****Riesgo 2 (BAJO):*** La CSP via `<meta>` no soporta todas las directivas (ej. `frame-ancestors`). Para CSP completa se requiere configuración a nivel de servidor HTTP. Mitigación: documentar en README que el servidor

de producción debe añadir el header HTTP correspondiente.

- **Riesgo 3 (MEDIO):** Google Analytics 4 requiere consentimiento de cookies (GDPR/aviso de cookies). La integración debe inicializarse SOLO después de que el usuario acepte. Se implementa con flag en localStorage.
- **Riesgo 4 (BAJO):** Sin HTTPS en entorno local el atributo `Secure` de cookies no aplica. Mitigación: el README debe especificar despliegue obligatorio en HTTPS (Netlify, GitHub Pages, Vercel son opciones gratuitas compatibles).

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO (integration-fix iter 1) | Iteraciones: 2
- assets/js/main.js | Estado: APROBADO (integration-fix iter 1) | Iteraciones: 1
- assets/css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 64,552

Tokens de salida (output): 34,530

Total tokens: 99,082

Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~16.7 horas-hombre ahorradas

"H squad de 4 personas × 2.1 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"910 input | !"1,433 output | 1 llamadas | ~1.5h ahorradas

MICROPLANIFICACION-DISEÑO: !"12,240 input | !"9,500 output | 4 llamadas | ~5.6h ahorradas

FRONTEND-DESARROLLO: !"32,544 input | !"15,856 output | 5 llamadas | ~5.1h ahorradas

test: !"18,858 input | !"7,741 output | 6 llamadas | ~4.4h ahorradas